Guía de Coloquio — Requisitos No Funcionales

Smart Learning · Sistema de gestión académica — Institución Cervantes

Tecnicatura Superior en Analista de Sistemas de Computación · Formación Especial I · División 5/1 NMS

Docente: Esp. Ing. Mariel Ferrando · Coloquio: 30/06/2026, 19:00 h · Integrantes: Faciolo Mayco, Amiel Ferreyra

Apertura (30 segundos)

"Smart Learning es un sistema web de gestión académica para la Institución Cervantes, construido en Next.js 16 con Supabase (Auth + PostgreSQL) y Drizzle ORM, desplegado en Vercel. Sobre la base de los requisitos funcionales que documentamos en Práctica Profesionalizante II, en esta etapa identificamos los requisitos no funcionales: las condiciones de calidad bajo las cuales el sistema debe operar, independientemente de qué funcionalidad esté ejecutando."

Tabla de requisitos (como en el PPT)

Categoría	Requisito No Funcional	Cómo se valida
Rendimiento	Páginas del dashboard cargan en ≤ 2 s (P95)	Vercel Speed Insights / Lighthouse
Seguridad	Tráfico cifrado HTTPS/TLS ≥ 1.2 y control de acceso por rol	SSL Labs (≥ A) y pruebas de acceso por rol
Disponibilidad	Uptime mensual ≥ 99,5%	Monitoreo con UptimeRobot
Confiabilidad	Backups automáticos diarios, retención ≥ 7 días	Verificación y prueba de restauración en Supabase
Usabilidad	Interfaz responsive desde 360 px de ancho	Pruebas en DevTools a 360/768/1280 px
Escalabilidad	Soporta ≥ 100 usuarios concurrentes	Prueba de carga con k6 / Artillery
Compatibilidad	Últimas 2 versiones de Chrome, Firefox, Edge y Safari	Pruebas funcionales por navegador (Playwright)
Mantenibilidad	Código TypeScript estricto, compila sin errores	`pnpm tsc --noEmit` en CI
Portabilidad	Configuración por variables de entorno	Despliegue en entorno limpio (preview de Vercel)
Aspectos legales	Cumple Ley 25.326 de Protección de Datos Personales (AR)	Auditoría del modelo de datos y consentimiento

Desarrollo por categoría — qué decir y con qué respaldarlo

Rendimiento — Páginas del dashboard cargan en ≤ 2 s (P95)

Por qué: docentes y administrativos cargan listados (alumnos, pagos, asistencia) varias veces por clase; la espera frena el uso diario.

> **Si preguntan "¿ya lo miden en producción?":** Speed Insights aún no está instalado en `package.json`. Es el método de validación *propuesto*, no un dato ya medido. Respuesta honesta: "definimos Lighthouse como herramienta de validación; instrumentarlo es el próximo paso."

Seguridad — HTTPS/TLS ≥1.2 + control de acceso por rol

> `src/proxy.ts` (Edge Runtime, reemplaza middleware.ts en Next 16) valida la sesión con `supabase.auth.getUser()` en cada request a rutas protegidas (/director, /administrativo, /profesor, /alumno), y bloquea al rol `administrativo` de secciones exclusivas de dirección (usuarios, docentes, cursos, marketing) redirigiendo a /403. Además, cada Server Action llama `requireRole()` como primera línea — defensa en profundidad: si algo bypasea el proxy, la mutación igual se corta.

HTTPS/TLS lo provee Vercel por defecto en todo dominio custom.

Disponibilidad — Uptime mensual ≥ 99,5%

Por qué: el sistema se usa en horario de clases; una caída implica no poder tomar asistencia o cargar notas ese día.

> El monitoreo con UptimeRobot es la herramienta propuesta, todavía no configurada — aclarar si preguntan.

Confiabilidad — Backups automáticos diarios, retención ≥ 7 días

Depende del plan de Supabase (point-in-time recovery / backups diarios son una feature de infraestructura del proveedor, no de código propio). Mencionarlo como decisión de elegir Supabase justamente porque resuelve esto "out of the box".

Usabilidad — Interfaz responsive desde 360 px

> Convención obligatoria del proyecto — tablas con `overflow-x-auto`, columnas secundarias `hidden md:table-cell`, sidebar como drawer responsive en mobile (`Sidebar.tsx`). Aplicado en todos los componentes, no es algo aislado.

Escalabilidad — ≥ 100 usuarios concurrentes

> El cliente Drizzle (`src/lib/drizzle/index.ts`) usa `max: 1` conexión por invocación serverless con `prepare: false` — esto es justamente lo que permite escalar: cada función de Vercel abre una sola conexión liviana al pooler de Supabase en modo transacción (puerto 6543), que multiplexa miles de conexiones cortas. La concurrencia real la maneja el pooler, no el proceso Node individual.

> **Pregunta trampa:** "¿max:1 no es una limitación?" → Es por diseño serverless: no se reutiliza una conexión persistente entre requests, así que max:1 evita abrir conexiones de más por invocación.

Compatibilidad — Últimas 2 versiones de Chrome, Firefox, Edge, Safari

> Tests e2e con Playwright (`tests/e2e/`) configurados con `devices["Desktop Chrome"]` , corriendo en CI (`.github/workflows/e2e-daily.yml`). Extensible a más navegadores fácilmente porque Playwright los soporta nativamente.

Mantenibilidad — TypeScript estricto, compila sin errores

> `tsconfig.json` tiene `"strict": true` . El CI (`.github/workflows/ci.yml`) corre `pnpm tsc --noEmit` en cada push — un solo error de tipos hace fallar el pipeline. 100% verificable.

Portabilidad — Configuración por variables de entorno

> Todo el acceso a Supabase, Resend y la DB sale de env vars (`NEXT_PUBLIC_SUPABASE_URL` , `DATABASE_URL` , `RESEND_API_KEY` , etc.), nunca hardcodeado. Permite desplegar en cualquier entorno (preview de Vercel, otro proyecto Supabase) sin tocar código.

Aspectos legales — Ley 25.326 (Protección de Datos Personales, Argentina)

Por qué (punto fuerte para "priorización crítica"): el sistema almacena datos de **menores de edad** (alumnos), DNI y datos de pagos — categorías sensibles bajo la ley argentina. Justifica priorizar seguridad y aspectos legales por sobre, por ejemplo, escalabilidad: el instituto es chico, pero el daño reputacional/legal de una filtración de datos de menores es alto independientemente del tamaño.

Cierre (conclusiones del slide 3)

- **Calidad integral:** los no funcionales no compiten con los funcionales, los completan.
- **Priorización crítica:** seguridad, disponibilidad y legales priorizados por gestionar datos de menores, académicos y de pagos.
- **Medición y verificabilidad:** cada requisito formulado de forma medible, evitando ambigüedad.
- **Arquitectura facilitadora:** Next.js + Vercel + Supabase + Drizzle resuelve varios atributos nativamente (HTTPS, backups, pooling, portabilidad) en vez de construirlos desde cero.

Pregunta trampa más probable: "¿Cómo saben que cumple eso de ≤2s o 100 usuarios si no lo midieron?"

Respuesta (sin ponerse a la defensiva): "Definimos la herramienta y el umbral medible como parte del requisito — la consigna pide evitar requisitos ambiguos como 'rápido' o 'fácil de usar', así que el foco de este entregable es expresarlo de forma verificable. La medición efectiva es la siguiente etapa, una vez en producción con tráfico real." Esto demuestra que entendieron el objetivo del ejercicio (ingeniería de requisitos), no que mintieron sobre el estado actual.